COM S 3110 Exam 2 Fall 2025

First name	
Last name	
Student ID number	
Lecture section	
Recitation section	

Some Useful Equalities: For any positive integer n,

- $\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$
- $\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$
- $2^{\log_2 n} = n$
- $a^{\log_b n} = n^{\log_b a}, \forall a > 1, b > 1$
- $n^{n/2} \leq n! \leq n^n$
- $\log x^a = a \log x, \forall x > 0$
- $\log(a \times b) = \log a + \log b, \forall a > 0, b > 0$
- $\log(a/b) = \log a - \log b, \forall a > 0, b > 0$
- $\log_b a = \frac{\log_c a}{\log_c b}, \forall a > 0, b > 1, c > 1$
- $a + ar + ar^2 + ... + ar^{n-1} = \frac{a(r^n-1)}{r-1}, \forall r \neq 1$
- $1 + \frac{1}{2} + \frac{1}{2^2} + ... + \frac{1}{2^n} = 2(1 - \frac{1}{2^{n+1}})$
- $1 + 2 + 4 + ... + 2^n = 2^{n+1} - 1$
- $x^{a+b} = x^a \times x^b, \forall x \neq 0$
- $x^{ab} = (x^a)^b, \forall x \neq 0$

Proof by Contradiction

1. Assume the negation of the statement you want to prove.
2. Derive the contradiction.
3. This contradiction then negates the assumption.

Proof by Induction

1. **Base case:** Prove that the statement is true for the initial value $n = n_0$.
2. **Induction hypothesis:** Assume that the statement holds for some arbitrary integer $n = k \geq n_0$.
3. **Induction step:** Using the inductive hypothesis, prove that the statement also holds for $n = k + 1$.

Definition (Big-O): Let $f : \mathbb{N} \to \mathbb{N}$ and $g : \mathbb{N} \to \mathbb{N}$ be functions on the natural numbers. $f(n) \in O(g(n))$ if and only if

$$\exists c > 0, \exists n_0 > 0 \text{ such that } \forall n \geq n_0, f(n) \leq cg(n) \tag{1}$$

Data Structures: The following table summarizes the runtime of common operations across different data structures.

Operation	Dynamic Array	Linked List	Sorted Array	Binary Search Tree
Search	$O(n)$	$O(n)$	$O(\log n)$	$O(\log n)$
Add	$O(1)$ amortized $O(n)$ worst case	$O(1)$	$O(n)$	$O(\log n)$
Delete	$O(n)$	$O(n)$	$O(n)$	$O(\log n)$

Hashing and Hash Table: Assume chaining-based hash table with uniform hashing and constant expected load factor and ignoring the time to compute hash code. The *expected* runtime for Search/Add/Delete is $O(1)$. The worst case for Search/Delete is $O(n)$, while Add takes $O(1)$ time if duplicate checking is not required.

Binary Heap: The following analysis assumes that array index starts at 1.

- A complete binary tree where each node has at most two children.
- A node's key is $\leq$ (min-heap) or $\geq$ (max-heap) its children's keys.
- The height of a heap tree is $\lfloor \log n \rfloor$.
- The min and max number of elements in a heap of height h is 2^h and $2^{h+1} - 1$.
- The children of node k are at $2k$ and $2k + 1$. The parent of k is at $\lfloor k/2 \rfloor$.
- To build a heap, insert each element at position $n + 1$ and heapifyUp.
- To remove element $H[i]$, swap it with $H[n]$, shrink H, and fix the heap (by calling heapifyDown or heapifyUp).
- An array `A` can be converted into a heap in $O(n)$ time by calling `makeHeap(A)`.
- Add/ExtractMin takes $O(\log n)$ time. FindMin takes $O(1)$ time.

```
heapifyUp(H, i) {
  if i == 1
    return
  j = floor(i/2)
  if H[i] < H[j] {
    swap H[i] and H[j]
    heapifyUp(H, j)
  }
}
```

Runtime: $O(\log n)$

```
heapSort(A) {
  makeHeap(A)
  n = size(A)
  for i = n down to 2
    swap H[1] and H[i]
    n--
    heapifyDown(H, 1, n)
}
```

Runtime: $O(n \log n)$

```
heapifyDown(H, i, size) {
  if 2i > size
    return
  j = 2i
  if 2i < size and H[2i] > H[2i+1]
    j = 2i + 1
  if H[j] < H[i] {
    swap H[i] and H[j]
    heapifyDown(H, j, size)
  }
}
```

Runtime: $O(\log n)$

```
makeHeap(A) {
  n = size(A)
  for i = floor(n/2) down to 1
    heapifyDown(A, i, n)
}
```

Runtime: $O(n)$

Graph Representation:

- Adjacency List: Space Complexity: $O(|V| + |E|)$.
- Adjacency Matrix: Space Complexity: $O(|V|^2)$.

BFS: Runtime is $O(|V| + |E|)$, assuming adjacency list representation. Applications: finding connected components, finding shortest path, determining whether a graph is bipartite.

```
BFS(v):
  Initialize x.explored = false for all nodes x
  Initialize an empty queue Q
  Add v to Q (to the tail)
  v.explored = true
  v.layer = 0
  while Q is not empty {
    Pop u from head of Q
    for all neighbors w of u:
      if w.explored == false then {
        add w to tail of Q
        w.explored = true
        w.layer = u.layer + 1
      }
  }
```

1. (10 points) For each of the following application scenarios, **circle exactly one** data structure (or abstract data type) that is best suited for performing the required task, given

   - an optimization criterion that describes the operation(s) whose time complexity should be optimized, and
   - the relevant assumptions/conditions that the data structure must respect.

   Your choice should be based primarily on the **asymptotic time complexity** of the operations identified in the optimization criterion, while respecting all given conditions. No explanation is needed.

   1.1 (2 points) You are designing a system in which tasks arrive continuously, each with an assigned priority level. All pending tasks must be stored until they are processed, and at any time, the system must process the task with the highest priority first.

   Optimization Criterion: Optimize the asymptotic time complexity of the two key operations: (1) inserting a new task and (2) retrieving and removing the highest-priority task.

   Assumptions/Conditions: Tasks are only added or processed (no arbitrary deletions or random lookups). Space usage is not a concern.

   (a) Linked List

   (b) Dynamic Array

   (c) Heap

   (d) Hash Table

1.2 (2 points) You are implementing a spam detection system for incoming emails. Each email has a unique identifier number. The system must track all previously seen email identifiers to avoid processing duplicates.

Optimization Criterion: Optimize the asymptotic time complexity of the two key operations: (1) inserting new identifiers and (2) checking whether an identifier has been seen before.

Assumptions/Conditions: Identifiers are unique integers. The system only needs to test membership and insert new entries (no sorting or ordered traversal). Memory use should remain reasonable but is not the main concern.

(a) Linked List

(b) Dynamic Array

(c) Heap

(d) Hash Table

1.3 (2 points) You are designing a text editor that maintains a history of operations to support undo and redo functionality.

Optimization Criterion: Optimize the asymptotic time complexity of inserting and deleting recent operations, as well as sequential traversal through the history.

Assumptions/Conditions: The total number of operations changes frequently. Memory should grow and shrink dynamically as needed, without reserving unused space.

(a) Linked List

(b) Dynamic Array

(c) Heap

(d) Binary Search Tree

1.4 (2 points) You are storing a social network of n users. Each user has a unique index from 0 to $n-1$ and may have any number of friends.

Optimization Criterion: Minimize asymptotic time complexity of determining whether two users, given by indices i and j, are directly connected as friends.

Assumptions/Conditions: The network is static, i.e., no users or friendships are added or removed. Memory is abundant, so space usage is not a concern.

(a) A **dynamic array** of size n. Entry $[i]$ is a **linked list** storing the indices of user i's friends in increasing order.

(b) A **dynamic array** of size n. Entry $[i]$ is a **dynamic array** storing the indices of user i's friends in increasing order.

(c) A **two-dimensional dynamic array** of size $n \times n$. Entry $[i][j]$ stores a value 1 if user i is friends with user j, and 0 otherwise.

(d) A **dynamic array** of size equal to the number of friendships. Each entry is a 2-element dynamic array storing the indices of two users who are friends.

1.5 (2 points) You are again storing a social network of n users with unique indices from 0 to $n-1$. In this version, the network is **sparse** (each user has relatively few friends).

Optimization Criterion: Balance query speed with memory efficiency when determining whether two given users are directly connected as friends.

Assumptions/Conditions: The network is static (no additions or deletions). Space usage now matters and should remain proportional to the number of friendships.

(a) A **dynamic array** of size n. Entry $[i]$ is a **linked list** storing the indices of user i's friends in increasing order.

(b) A **dynamic array** of size n. Entry $[i]$ is a **dynamic array** storing the indices of user i's friends in increasing order.

(c) A **two-dimensional dynamic array** of size $n \times n$. Entry $[i][j]$ stores a value 1 if user i is friends with user j, and 0 otherwise.

(d) A **dynamic array** of size equal to the number of friendships. Each entry is a 2-element dynamic array storing the indices of two users who are friends.

2. (10 points) Consider the following **undirected** graph $G = (V, E)$:

$$\begin{aligned} V &= \{1, 2, 3, 4, 5\} \\ E &= \{\{1, 2\}, \{1, 3\}, \{2, 4\}, \{3, 4\}, \{4, 5\}\} \end{aligned}$$

(a) (5 points) Fill in the **adjacency matrix** representation of graph G below.

	1	2	3	4	5
1	0	1	1	0	0
2	1	0	0	1	0
3	1	0	0	1	0
4	0	1	1	0	1
5	0	0	0	1	0

(b) (5 points) Fill in the **adjacency list** representation of graph G below. Specifically, for each vertex, list its neighbors in **increasing order** of vertex number.

Vertex	Neighbors
1	2, 3
2	1, 4
3	1, 4
4	2, 3, 5
5	4

3. (10 points) Consider an array $A = [A[1], A[2], \ldots, A[n]]$ of n integers that satisfies the **min-heap** property.

   For each of the following modifications to A, determine the **tightest possible upper bound**, i.e., the **exact maximum number of swaps** that a correct and most efficient algorithm **requires in the worst-case scenario** to restore the heap property after the modification.

   For each modification, describe your answer by filling in all three parts below:

   - **Maximum number of swaps possible:** The largest number of swaps that a correct and most efficient algorithm must perform to restore the heap property in the worst-case scenario. Your answer must be expressed precisely in terms of n, i, and constants as appropriate. *Giving asymptotic (Big-O) bound will receive 0 points.*
   - **When is this maximum achieved:** Describe the specific configuration(s) of the heap and/or the change to A that produces exactly the number of swaps identified in the first bullet. Only configurations that arise from a correct algorithm count. Do not consider artificial or unnecessary swaps.
   - **Reason:** Briefly explain why this number of swaps is both necessary and sufficient in the worst-case scenario identified in the second bullet.

   (a) (3 points) Decrease the value of $A[1]$.

   - **Maximum number of swaps possible, in terms of n and constants:**
     0
   - **When is this maximum achieved:**
     For any configuration of A, decreasing $A[1]$ will never require a swap.
   - **Reason:**
     $A[1]$ is the root of the min heap so it has no parent. Thus, decreasing its value cannot violate the heap property with any parent.

(b) (3 points) Increase the value of $A[n]$.

- **Maximum number of swaps possible, in terms of n and constants:**
  0

- **When is this maximum achieved:**
  For any configuration of A, increasing $A[n]$ will never require a swap.

- **Reason:**
  $A[n]$ is a leaf node so it has no children. Thus, increasing its value cannot violate the heap property with any children.

(c) (4 points) Decrease the value of $A[i]$ for some $i \in \{1, 2, \ldots, n\}$.

- **Maximum number of swaps possible, in terms of n, i and constants:**
  $\lfloor \log i \rfloor$

- **When is this maximum achieved:**
  When the new value of $A[i]$ is smaller than the root $A[1]$

- **Reason:**
  Decreasing $A[i]$ may cause $A[i]$ to be smaller than its parent; thus, violating the heap property. In the worst case, the new value of $A[i]$ can be smaller than the root $A[1]$. Thus, to restore the heap property in this case, we need to bubble $A[i]$ up to the root. Since when we bubble up $A[i]$, the elements $A[i+1], \ldots, A[n]$ does not matter, we can think of this as the heap of i elements $\{A[1], A[2], \ldots, A[i]\}$. As discussed in class (and also on a cover page), the height of this heap is exactly $\lfloor \log i \rfloor$.

4. (10 points) Let $A = [A[1], A[2], \ldots, A[n]]$ be a dynamic array of integers. For any $i, j \in \{1, \ldots, n\}$ such that $i < j$, let

$$S_{ij} = A[i+1] + A[i+2] + \ldots + A[j]$$

denote the sum of the subarray starting at index $i+1$ and ending at indenx j.

Your task is to design an **efficient algorithm** that takes A and an integer K as input and determines whether there exist indices $i, j \in \{1, \ldots, n\}$ with $i < j$ such that

$$S_{ij} = K.$$

Complete the following subparts to answer this question. For each subpart, **circle your final answer**.

Hint: Consider using prefix sums in very much the same way as in the subset sum problem discussed in lecture.

(a) (3 points) For each $k \in \{1, \ldots, n\}$, let

$$P[k] = A[1] + A[2] + \ldots + A[k] \tag{2}$$

be the prefix sum of the first k elements of A as defined in class.

Consider arbitrary indices $i, j \in \{1, \ldots, n\}$ with $i < j$. State a **necessary and sufficient condition, in terms of** $P[i]$, $P[j]$ **and** K, such that $S_{ij} = K$.

Consider arbitrary indices $i, j \in \{1, \ldots, n\}$ such that $i < j$.

$$\begin{aligned} P[i] &= A[1] + A[2] + \ldots + A[i] \\ P[j] &= A[1] + A[2] + \ldots + A[i] + A[i+1] + \ldots + A[j] \end{aligned}$$

So

$$P[j] - P[i] = A[i+1] + A[i+2] + \ldots + A[j] = S_{ij}.$$

Thus, $S_{ij} = K$ if and only if

$$P[j] - P[i] = K$$

(b) (3 points) Consider an arbitrary index $j \in \{2, \ldots n\}$. Suppose we have already computed the prefix sums

$$P[1], P[2], \ldots, P[j-1],$$

where $P[k]$ is defined in Equation (2) in part (a) for each $k \in \{1, \ldots, j-1\}$.

State, in terms of $A[1], \ldots, A[n], P[1], \ldots, P[j-1]$ and K, the **value that must appear among the prefix sums** $P[1], P[2], \ldots, P[j-1]$ if and only if there exists $i \in \{1, \ldots, j-1\}$ such that $S_{ij} = K$.

Your answer must be **computable in** $O(1)$ **worst-case time** for any $j \in \{2, \ldots, n\}$. You can assume that accessing any element $A[1], \ldots, A[n], P[1], \ldots, P[j-1]$ takes $O(1)$ time.

From part (a), we have $S_{ij} = K$ if and only if

$$\begin{aligned} P[j] - P[i] &= K \\ P[i] &= P[j] - K \end{aligned}$$

Thus, for a given j, we need to check whether P contains $P[j] - K$. Since $P[j] = P[j-1] + A[j]$, this condition is equivalent to checking whether

$$P[j-1] + A[j] - K$$

appears among the prefix sums $P[1], P[2], \ldots, P[j-1]$. This computation requires only basic arithmetic on three numbers, regardless of the value of j and can therefore be performed in $O(1)$ worst case time for any given j.

(c) (2 points) From part (b), we know that for each index $j \in \{2, \ldots, n\}$, the algorithm must check whether a certain value appears among the previously computed prefix sums $P[1], P[2], \ldots, P[j-1]$.

List **all the main operations** that must be supported by the data structure used to maintain these prefix sums $P[1], P[2], \ldots, P[j-1]$ across all values of index j, so that the algorithm remains efficient. For each operation, briefly explain why it is necessary.

We need two main operations

1. insertion: to add a new prefix sum $P[j]$ to P.
2. search: to check whether $P[j] - K$ appears in $P[1], P[2], \ldots, P[j-1]$.

(d) (2 points) Given the operations you identified in part (c), identify an appropriate data structure to maintain the prefix sums $P[1], P[2], \ldots, P[j-1]$ across all values of index j. Also state the time complexity of each operation. No justification is needed.

Hash table

Both insertion and search take $O(1)$, **expected** time.

5. (10 points) A university is organizing a large-scale academic conference and has invited n experts, $x_1, \ldots, x_n$, to speak on various topics.

   The organizers must assign these experts into **two separate sessions**. However, some experts have personal conflicts and refuse to be placed in the same session. These conflicts are represented by a list C of expert pairs (i, j), where $i, j \in \{1, \ldots, n\}$ and $i \neq j$. Each pair $(i, j) \in C$ indicates that experts x_i and x_j must be assigned to different sessions.

   (a) (2 points) State the name of the **abstract graph problem** (e.g, reachability, connected components, cycle detection, topological sorting, bipartite graph check, shortest path) to be solved that best models this scenario. In other words, identify the type of graph problem you would solve if you represent experts as vertices and conflicts as edges. No explanation needed.

   Bipartite graph check

   (b) (2 points) Which data structure is best suited for storing the experts and conflict pairs for this problem? Here, we only care about asymptotic time complexity for solving the entire problem, including both constructing the representation (considering amortized costs for dynamic arrays) and running the algorithm to solve it. **Select all that apply**. No explanation needed.

   (a) A **dynamic array** of size n. Entry $[i]$ is a **linked list** storing all indices j such that $(i, j) \in C$ or $(j, i) \in C$.

   (b) A dynamic array of size n. Entry $[i]$ is a **dynamic array** storing all indices j such that $(i, j) \in C$ or $(j, i) \in C$.

   (c) A **two-dimensional dynamic array** of size $n \times n$. Entry $[i][j]$ stores a value 1 if user i is friends with user j, and 0 otherwise.

   (d) A **dynamic array** of size equal to the number of expert pairs in C. Each entry is a 2-element dynamic array storing the indices of two expert pairs in C.

(c) (3 points) Write the **tightest asymptotic (Big-O) runtime** for each of the following tasks **in terms of the number of experts, n, and the number of conflict pairs** $|C|$. For each subpart, **circle your final answer**. No explanation needed.

- Construct your chosen representation in part (b):
  $O(n + |C|)$

- Solve the problem in part (a) using your representation in part (b):
  $O(n + |C|)$

- Overall runtime (combine the two):
  $O(n + |C|)$

(d) (3 points) Suppose the conflict pairs form several groups that have conflicts only inside each group and never between groups. Formally, the experts can be **partitioned** into subsets $X_1, X_2, \ldots, X_m$ satisfying

1. For all $k \in \{1, \ldots, m\}$, X_k is a subset of experts, i.e.,

$$X_k \subseteq \{x_1, x_2, \ldots, x_n\}$$

2. The subsets $X_1, X_2, \ldots, X_m$ are disjoint, i.e.,

$$\forall k \neq k', X_k \cap X_{k'} = \emptyset$$

3. Together these subsets cover all experts, i.e.,

$$X_1 \cup X_2 \cup \ldots \cup X_m = \{x_1, x_2, \ldots, x_n\}$$

4. Every conflict pair lies entirely within one subset, i.e.,

$$\text{if } (i,j) \in C, \text{ then there exists } k \in \{1, 2, \ldots, m\} \text{ such that both } x_i, x_j \in X_k.$$

In other words, each subset X_k is an independent conflict group.

Question: Describe whether knowing the groups $X_1, X_2, \ldots, X_m$ can improve the **time complexity** of your algorithm. If yes, describe how. If not, explain why.

Knowing $X_1, X_2, \ldots, X_m$ does **NOT** change the asymptotic time complexity, which remains $O(n + |C|)$.

Reason: Even if we process each group independently, the total number of vertices and edges across all groups is still n and $|C|$. The BFS algorithm runs in $O(n + |C|)$, whether we handle one large graph or multiple disjoint subgraphs. Thus, the asymptotic runtime remains $O(n + |C|)$.

Question	Points	Score
1	10	
2	10	
3	10	
4	10	
5	10	
Total:	50	